

Міністерство освіти і науки України
Державний університет „Житомирська політехніка”

Кафедра КТуМтаТ

Група: ТР-25-1

Теорія ймовірностей та математична статистика
Практична робота №2
«Комбінаторні моделі та розрахунок ймовірностей у програмних системах»

Перевірів:

Лисенко М. С.

Виконав:

Наумчук А. В.

					ІКТР.420001.123-ЗЛ			
Змн.	Арк.	№ докум.	Підпис	Дата	Звіт з практичної роботи	Літ.	Арк.	Аркушів
Розроб.		Наумчук А. В.					1	
Перевір.		Лисенко М. С.						
Керівник						ФІКТ, гр. ТР-25-1		
Н. контр.								
Затверд.								

Мета: засвоїти практичні навички застосування комбінаторного аналізу (перестановки, розміщення, комбінації, правила суми та добутку) для обчислення ймовірностей у дискретних просторах елементарних подій; навчитися симулювати комбінаторні об'єкти, оцінювати ймовірності колізій, конфігурацій та атак на програмні системи.

Теоретична модель:

1. У цьому блоці моделюється ймовірність виникнення хоча б однієї колізії при генерації k випадкових ідентифікаторів у просторі розміром $N = 365$. За правилом добутку, кількість способів обрати k унікальних елементів (без повторень) з N дорівнює кількості розміщень $A(N, k)$. Загальна кількість усіх можливих комбінацій з повтореннями становить N^k . Ймовірність того, що всі k елементів будуть унікальними (відсутність колізій), обчислюється як:

$$P(\text{без колізії}) = \frac{A(N, k)}{N^k}$$

Тоді

$$P(\text{колізія}) = 1 - \frac{A(N, k)}{N^k}$$

2. Генерується пароль із 6 символів без повторень. пароль складається з 10 цифр та 26 літер (англійський алфавіт), (загалом $n = 36$ символів). Оскільки порядок символів у паролі має значення, загальний простір елементарних подій Ω обчислюється через формулу розміщень:

$$|\Omega| = A(36, 6) = \frac{36!}{(36 - 6)!}$$

3. Випадкова точка (x, y) рівномірно розподілена в одиничному квадраті $[0, 1] \times [0, 1]$. Її площа $S_{\text{квадрат}} = 1$. Область D обмежена кривими $y \leq x^2$ та $y \geq 1 - x$. Для знаходження площі області D спочатку визначається точка перетину кривих, прирівнюючи їхні праві частини:

					ІПТР.420001.123-ЗЛ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		2

$$x^2 = 1 - 1 \Rightarrow x^2 + x - 1 = 0$$

4. Система має 4 параметри конфігурації, кожен з яких може набувати 3 значень. Загальна кількість можливих конфігурацій (простір подій) обчислюється за правилом добутку:

$$N = 3^4$$

Практична частина

БЛОК 1:

Таблиця 1.1

k	P(теор)	W(стат)	похибка
5	0.02714	0.02930	0.00216
10	0.11695	0.11770	0.00075
15	0.25290	0.25250	0.00040
20	0.41144	0.41640	0.00032
23	0.50730	0.50710	0.00042
30	0.70632	0.69960	0.00034
40	0.89123	0.89090	0.00036
50	0.97037	0.96950	0.00039
60	0.99412	0.99250	0.00041
70	0.99916	0.99930	0.00064
80	0.99991	1.00000	0.00009

C:\Users\asusp\AppData\Local

Блок 1: Дослідження колізії

k	P(теор)	W(стат)	похибка
5	0.02714	0.03020	0.00306
10	0.11695	0.11590	0.00105
15	0.25290	0.25210	0.00080
20	0.41144	0.40400	0.00744
23	0.50730	0.51180	0.00450
30	0.70632	0.70590	0.00042
40	0.89123	0.89160	0.00037
50	0.97037	0.97070	0.00033
60	0.99412	0.99370	0.00042
70	0.99916	0.99980	0.00064
80	0.99991	1.00000	0.00009

Press any key to continue . . .

Рис1.1. Вихідні дані виконання програми у вигляді «таблиці»

```

for k in k_values:
    p_no_collision = 1.0
    for i in range(k):
        p_no_collision *= (1.0 - i / N_space)
    p_theor = 1.0 - p_no_collision

    collisions = 0 #Симуляція методом монте карло
    for _ in range(simulations):
        items = [random.randint(1, N_space) for _ in range(k)] #генерація рандомних чисел від 1 до 365
        if len(set(items)) < len(items):
            collisions += 1

    w_stat = collisions / simulations#вирахування статистичної частоти
    error = abs(w_stat - p_theor)

```

Рис1.2. Частина коду яка виконує основні обрахунки

БЛОК 2

Блок 2: парольна безпека (варіант 1)

подія	теорія	статистика	похибка
P(A)	0.11820	0.11935	0.00115
P(B)	0.02303	0.02362	0.00059
P(A ∩ B)	0.00000	0.00000	0.00000
P(A ∪ B)	0.14123	0.14297	0.00174
P(не A)	0.88180	0.88065	0.00115

Press any key to continue . . .

Рис2.1 Вихідні дані виконання програми у вигляді «таблиці»

```
# Теоретичні розрахунки
total_passwords = math.perm(36, 6)
p_A_theor = math.perm(26, 6) / total_passwords
p_B_theor = (math.perm(10, 2) * math.perm(26, 4)) / total_passwords
p_A_intersect_B_theor = 0.0 #пароль не може бути тільки з букв та мати 2 цифри
p_A_union_B_theor = p_A_theor + p_B_theor
p_not_A_theor = 1.0 - p_A_theor
```

Рис2.2 Теоретичний розрахунок за допомогою коду

```
for i in range(simulations):
    pwd = random.sample(pool, 6)#генерація унікальної комбінації
    is_A = all(c == 'L' for c in pwd)#пароль тільки з букв
    is_B = (pwd[0] == 'D' and pwd[1] == 'D' and #перші буква потім 4 цифри
            pwd[2] == 'L' and pwd[3] == 'L' and
            pwd[4] == 'L' and pwd[5] == 'L')
```

Рис2.3 Генерація унікальної комбінації яка не повторюється, та перевірка

БЛОК 3

```
===== Блок 3: геометрична ймовірність складної області =====
N          | W(D) (стат)      | S(D) (теор)      | похибка
-----|-----|-----|-----
100         | 0.200000         | 0.181695         | 0.018305
1000        | 0.174000         | 0.181695         | 0.007695
10000       | 0.185000         | 0.181695         | 0.003305
100000      | 0.181360         | 0.181695         | 0.000335
1000000     | 0.181980         | 0.181695         | 0.000285
```

Рис3.1 Вихідні дані виконання програми у вигляді «таблиці»

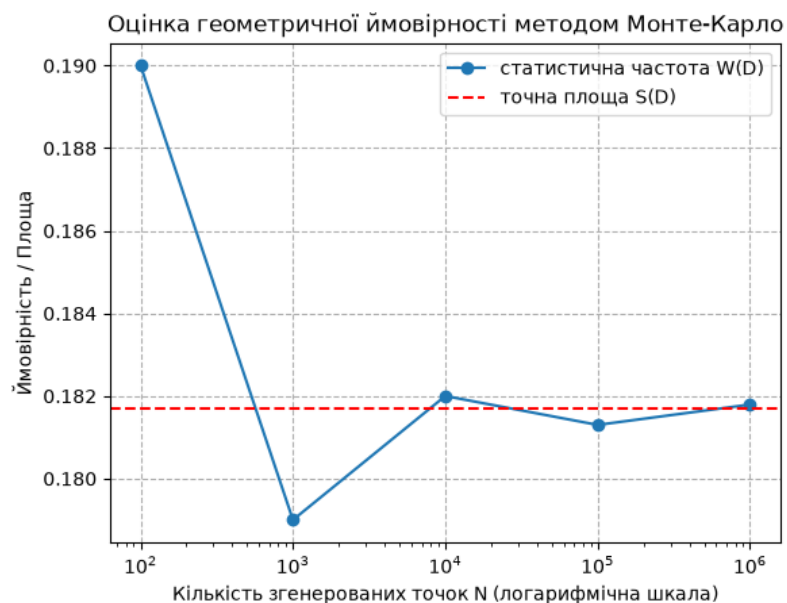


Рис3.2 Графік оцінки гео. ймовірності методом Монте-Карло

```

for n in N_values:
    matches = 0
    for _ in range(n):
        x = random.random()
        y = random.random()
        if y <= x**2 and y >= 1 - x: # перевірка потрапляння точки в задану область
            matches += 1

```

Рис3.3. Частина коду яка перевіряє потрапляння точки в задану область

БЛОК 4

```

===== Блок 4: тестове покриття =====
загальна кількість конфігурацій: 81
загальна кількість тестового набору: 9
теоретична ймовірність: P = 9/81 = 0.1111

проведенно симуляцій: 10000
потрапляння в тестову множину: 1120
статистична частота: W = 0.1120
абсолютна похибка: 0.0009
Press any key to continue . . . |

```

Рис4.1 Вихідні дані виконання програми

```

p_theor = test_subset_size / total_configurations
print(f"загальна кількість конфігурацій: {total_configurations}")
print(f"загальна кількість тестового набору: {test_subset_size}")
print(f"теоретична ймовірність: P = {test_subset_size}/{total_configurations} = {p_theor:.4f}\n")

hits = 0
for i in range(simulations):
    random_config = random.randint(1, total_configurations)
    if random_config <= test_subset_size:
        hits += 1

```

Рис4.2 Частина коду яка виконує теоретичний розрахунок та цикл симуляції.

Посилання на репозиторій

https://github.com/8sepp8/PR2_PoT_Naumchuk_TR-25-1

					IPTP.420001.123-3Л	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		6

ВИСНОВОК

Під час виконання другої практичної роботи застосував комбінаторний аналіз та метод Монте-Карло для моделювання випадкових подій у програмних системах. Написані на Python , скрипти наочно підтвердили дію Закону великих чисел: на великих вибірках (до 100 000 ітерацій) статистичні частоти для задач із колізіями, генерацією паролів, обчисленням площі геометричної області та оцінкою Pairwise-тестуванням.

					ІТР.420001.123-ЗЛ	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		